作品名　SUPERCHUCHUODYSSEY
(スーパーチュウチュウオデッセイ)

ジャンル　3Dアクションゲーム

開発環境　C++,DxLib

対応機種　windows

制作期間　3か月

制作時期　2026年6月～

制作人数　1人

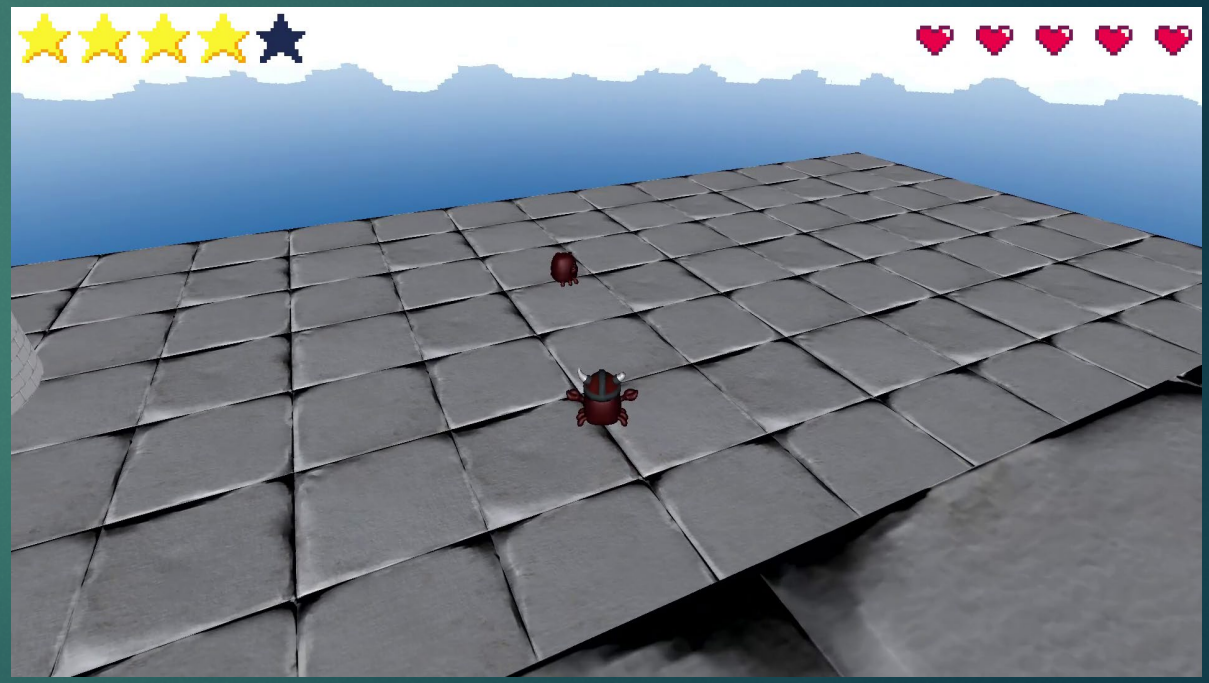


ゲーム概要

マップに散らばった星を集める
3Dアクションゲーム



帽子を使うことで敵の体を乗っ取れる
敵独自のアクションが可能になる



技術紹介①
インターフェース

いろんなクラスで共通して持たせたい
機能が複数あった

⇩

インターフェースとして定義することで
継承するだけで機能を使えるように

⇩

プレイヤー、敵などで処理の共通化ができた

新しいオブジェクトの追加がしやすい

```
/// <summary>
/// 操作することができるオブジェクトの抽象基底クラス
/// </summary>
class IControllable
{
public:
    virtual ~IControllable() = default;
    //入力をセットする
    void SetInputInfo(const InputInfo& input) { m_input = input; }
    //入力を取得する
    InputInfo GetInputInfo()const { return m_input; }
    //移動
    virtual void Move() abstract;
    //ジャンプ
    virtual void Jump()abstract;
    //上昇を切る
    virtual void JumpEnd()abstract;

    //操作対象になったときに呼ばれる
    virtual void Controll()abstract;
    //操作対象から外れたときに呼ばれる
    virtual void
protected:
    InputInfo m_
};
```

```
class CollisionManager;
class ICollider
{
public:
    virtual ~ICollider() = default;

    virtual const Collider& GetCollider()const abstract;
//  virtual const Ray& GetRay()const abstract;

    /// <summary>
    /// 自身のコリジョンレイヤーを返す
    /// </summary>
    /// <returns></returns>
    virtual CollisionLayer GetCollisionLayer()const abstract;
    /// <summary>
    /// 当たり判定をするコリジョンレイヤーを返す
    /// </summary>
    /// <returns></returns>
    virtual CollisionLayer GetCollisionMask()const abstract;
    /// <summary>
    /// 当たった
    /// </summary>
    /// <param name="other"></param>
    virtual void OnCollision(ICollider& other,CollisionResult& result) abstract;
};
```

技術紹介②
操作対象の切り替え

操作するキャラクターを切り替えたい

⇩

操作対象をターゲットとして保存して
入力を渡すマネージャーを作成

⇩

操作する側と操作される側で分けることで
操作対象の切り替えがやりやすくなっている

```cpp
void PlayerController::Update()
{
    //入力取得
    auto& input = InputManager::GetInstance();
    //操作対象のオブジェクトをターゲットにする
    m_target = m_target->GetControllTarget();

    //ターゲットに入力を渡す
    InputInfo inputInfo;
    inputInfo.move = input.GetLeftStick();
    inputInfo.isJumpDown = input.IsTriggered("A");
    inputInfo.isJumpUp = input.IsReleased("A");
    inputInfo.isHipDropDown = input.IsTriggered("B");
    m_target->SetInputInfo(inputInfo);
```

```cpp
void PlayerController::SetTarget(ICaptureTarget* target)
{
    if (m_target)
    {
        //前のターゲットのExitを呼ぶ
        m_target->ExitControll();
    }
    //次のターゲットをセット
    m_target = target->GetControllTarget();
    m_target->Controll();
}
```